



EAD

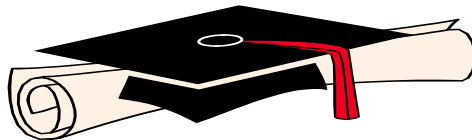
Enterprise Application Development in JakartaEE

Semester: 4

The Exam Code: 02

Duration: 90 minutes

Total Marks: 20



Practical Examination Paper

Do not write on this question paper and return it to the Invigilator after the examination.

IDE: IntelliJ IDEA or Visual Studio Code

Server: WildFly (or GlassFish/Payara)

Project Scenario: You are required to develop the core backend for a simplified Banking System using Jakarta EE technologies to manage user sessions and transactions.

Question 1: Enterprise Beans, CDI, and Validation (12 Points) Create a Jakarta EE Web Profile project and implement the following requirements:

1.a (3 Points) - POJO & Validation: * Create a **BankAccount** class with fields: `accountNumber` (String), `customerName` (String), and `balance` (double).

- Generate a default constructor, a parameterized constructor, getters, setters, and a `toString()` method for the class.
- Apply Jakarta Bean Validation constraints:
 - `accountNumber`: Cannot be null and must be exactly 10 characters long.
 - `balance`: Must be greater than or equal to 0 (Zero).

1.b (4 Points) - Interceptor & CDI: * Create a **@Stateless** bean named **NotificationService** with a method `sendAlert(String message)`. This method should print the message to the server console (e.g., "ALERT: [message]").

- Create an Interceptor binding and its implementation class (e.g., **@AuditTransaction**) that logs the text "Transaction process started..." to the server console before the target method is executed.

1.c (5 Points) - Stateful Session Bean: * Create a **@Stateful** session bean named **BankingSessionBean** (with a Local interface **BankingSessionLocal**).

- This bean must maintain a `BankAccount` object representing the currently logged-in user's account state.
- Implement a `login(BankAccount account)` method to initialize the state.
- Implement a `deposit(double amount)` method to add money to the balance. You must validate that the amount is greater than 0 (throw an exception or reject if `amount <= 0`). Apply the **@AuditTransaction** interceptor to this method.
- Use **@Inject** to inject the `NotificationService` into your `BankingSessionBean` and call `sendAlert()` whenever a successful deposit is made.

Question 2: Transactions and Servlet Client (8 Points) Integrate transaction management to handle fund transfers and test your EJBs using a Web Client:

2.a (3 Points) - Container-Managed Transactions (CMT): * Add a void transferFunds(double amount, String targetAccount) method to your **BankingSessionBean**.

- Configure this method using the **@TransactionAttribute(TransactionAttributeType.REQUIRED)** annotation.
- Inside the transferFunds() method:
 - Validate that the amount is greater than 0. If amount <= 0, reject the transaction (throw an exception).
 - Check if the current balance is less than the amount. If yes, throw a RuntimeException with the message "Insufficient funds!" (This will trigger a transaction rollback).
 - If sufficient and valid, deduct the amount from the balance and print "Successfully transferred to [targetAccount]" to the console.

2.b (5 Points) - Servlet Client: * Create a Jakarta Servlet named **BankingController** mapped to the URL /bank.

- Inject the **BankingSessionLocal** bean into this servlet using **@EJB** or **@Inject**.
- In the doGet method, simulate a user workflow:
 - Instantiate a valid BankAccount object with an initial balance of 500.0.
 - Call the login() method with this account.
 - Call the deposit(200.0) method.
 - Call the transferFunds(100.0, "0987654321") method.
 - Print the final account details and balance to the web browser response (PrintWriter).

Grading Rubric (For Instructor)

- **Q1:** Correct BankAccount validation, constructors, getters, setters (3 pts). NotificationService is stateless & Interceptor is created properly (4 pts). BankingSessionBean is stateful, holds account state, handles valid deposit logic (>0), and CDI injection works (5 pts).
- **Q2:** Correct transaction annotation, >0 amount check, and rollback logic on insufficient funds (3 pts). Servlet successfully simulates the workflow without errors and outputs correct final balance (600.0) on the browser (5 pts).